

Software Simulation of an Autonomous Wildfire Early Warning System

Edge-Fog-Cloud Architecture — Simulation Ideation Document (v1)

Parent concept: Autonomous Wildfire Early Warning and Pre-Suppression Using Edge-Fog-Cloud Architecture in Indian Forest Ecosystems This document defines: A containerized software simulation of that architecture, built to validate the decision logic, coordination behavior, and offline-resilience claims of the original design before any physical hardware is procured or deployed.

1. Purpose and Scope

The parent ideation document proposes a three-tier Edge-Fog-Cloud (EFC) system with physical sensors, LoRa radios, solar-powered fog servers, and actuated pre-suppression hardware. Building that system directly carries real cost and risk: procurement lead time, field deployment logistics, and the difficulty of testing failure modes (e.g., a real wildfire) safely.

This document scopes a **pure-software simulation** of the same architecture. The simulation reproduces the *logic and data flow* of the system — sensor behavior, fog-level decision-making, inter-node coordination, and cloud aggregation — inside Docker containers on a single development machine or a small cluster. It is explicitly a **decision-architecture validation tool**, not a hardware substitute.

1.1 Why simulate first

- Validates the risk-scoring model and state machine (GREEN/YELLOW/ORANGE/RED) against synthetic data before it has to work on real sensors.
- Lets failure scenarios (network loss, sensor faults, simultaneous multi-zone escalation) be tested repeatedly and safely.
- Produces measurable evidence for the latency and coordination claims made in the original paper (Sections 7.1, 6.3) — evidence a hardware-only approach can't easily produce early.
- Gives you a working, demonstrable artifact for review/evaluation before any capital is spent on sensors, fog servers, or actuators.

- De-risks the eventual hardware build: interface contracts (message schemas, topic structure, state transitions) get defined and tested in software first, then hardware is built to match a known-working spec.

1.2 Relationship to the original document

Every simulated component maps to a real component in the parent paper. Nothing new is being invented architecturally — this document is an **implementation plan for a testbed**, not a redesign. Section 11 gives the explicit mapping and marks what is faithfully simulated, what is approximated, and what is entirely out of scope.

2. Objectives

1. Reproduce the edge → fog → cloud data flow using synthetic sensor generators in place of physical hardware.
 2. Implement the composite, multi-parameter risk-scoring model (Section 4.3.2 of the parent paper) as real, runnable rule-based logic.
 3. Implement the four-state classification (GREEN/YELLOW/ORANGE/RED) with the multi-sensor confirmation rule (Section 8.4) to reduce false positives.
 4. Implement bounded "autonomous actions" as logged, simulated commands (no real actuators).
 5. Implement lateral fog-to-fog coordination (Section 6.3) — one zone's escalation influencing a downstream zone's state.
 6. Implement and test offline-resilience: fog nodes must continue operating and taking bounded action when the simulated cloud link is cut.
 7. Provide a scenario injection harness to drive the system through realistic and edge-case sequences (normal day, slow-building risk, sudden ignition, sensor fault, cloud outage during an active event).
 8. Provide a dashboard for observing system state, decision logs, and timing metrics in real time.
 9. Produce quantitative results (decision latency, false-positive rate under fault injection, lateral-alert propagation time) that can be reported as evidence for the architecture's viability.
-

3. Simulated System Overview

The simulation preserves the three tiers, with software surrogates replacing physical elements:

Original tier	Original component	Simulated equivalent
Edge	Weatherproof sensor cluster + LoRa radio	Containerized Python process emitting synthetic sensor readings on a schedule, publishing over MQTT
Edge↔Fog link	LoRa (868 MHz), star + mesh fallback	MQTT topics over the Docker network, with an optional network-impairment layer (latency, jitter, packet loss) to approximate LoRa's constraints
Fog	Jetson Orin NX / ARM edge server	Containerized service per forest zone, running the risk-scoring engine and state machine
Fog↔Fog link	4G LTE mesh / sub-GHz WiFi	MQTT topics on a shared "regional" channel, or direct service-to-service calls
Fog↔Cloud link	4G LTE / VSAT	MQTT/HTTP to the cloud broker, interruptible on demand to simulate connectivity loss
Cloud	MeghRaj / state data center	Containerized broker + time-series database + dashboard
Actuators (mist systems, drones, acoustic deterrents)	Physical hardware	Logged, simulated "action" events only — no physical or virtual actuator control
Control center	Forest Range Office	A simple alert feed/dashboard panel representing what an RFO would see

4. Detailed Component Design

4.1 Edge Simulation Layer

Container: `edge-sim` (one instance per simulated edge node, parameterized by node ID and zone).

Responsibilities:

- Generate synthetic values for each sensor in the original sensor stack (Section 4.2.1): temperature, humidity, wind speed/direction, soil moisture, gas/smoke level, thermal anomaly reading, ambient light, rainfall, and GPS coordinate (static per node).
- Support three data-generation modes, selectable at runtime:
 - **Baseline mode** — values drift randomly within normal seasonal ranges.
 - **Scenario-driven mode** — values follow a predefined trajectory supplied by the scenario injector (Section 4.6), e.g., a slow rise in temperature and fall in humidity over 20 minutes to simulate a drying trend.
 - **Fault mode** — a sensor "sticks," drops out, or reports an out-of-range value, to test the fog layer's fault-tolerance logic (Section 8.4).
- Publish readings to the MQTT broker on a per-node topic (e.g., `zone/{zone_id}/edge/{node_id}/reading`) at a configurable interval (default every 5 minutes, increasing to every 30 seconds under simulated elevated risk — mirroring Section 5.2).
- Attach a monotonic timestamp and sequence number to every reading, to support latency measurement downstream.

4.2 Edge–Fog Communication Layer

Container: `mqtt-broker-local` (Eclipse Mosquitto), one per simulated forest division, acting as the local LoRa-gateway surrogate.

- Star topology by default: all edge nodes in a zone publish to their zone's broker.
- An optional **impairment sidecar** (using `tc netem` on the broker container's network interface) can inject latency (tens of ms to a few seconds), jitter, and packet loss to approximate LoRa's real-world unreliability in forested terrain. This is clearly labeled as an approximation, not a physical-layer model.
- Topic naming convention documented in Section 6.

4.3 Fog Node Simulation — "Local Brain"

Container: `fog-node` (one instance per simulated forest zone/division).

This is the core logic of the simulation and the primary object of validation.

Step 1 — Ingestion. Subscribes to all edge topics for its zone; maintains a rolling in-memory buffer (e.g., last 30 minutes of readings per sensor).

Step 2 — Composite risk scoring. Implements the weighted scoring function described conceptually in Section 4.3.2 of the parent paper, as an explicit, auditable rule set:

$$\text{risk_score} = w1f(\text{temperature}) + w2f(\text{humidity}) + w3f(\text{wind_speed})$$
$$+ w4f(\text{soil_moisture}) + w5f(\text{gas_level}) + w6f(\text{thermal_anomaly})$$
$$+ w7f(\text{time_of_day}) + w8f(\text{seasonal_baseline})$$

- Weights (`w1 . . w8`) and normalization functions (`f`) are configurable via a YAML/JSON config file per zone, so different simulated forest types (dry deciduous vs. moist Himalayan, for example) can be tested without code changes.
- Initial implementation is fully rule-based and transparent (per the parent paper's stated design philosophy in 4.3.2) — no ML model in the simulation's first phase. A stub interface is defined so a lightweight ML classifier (e.g., a small Random Forest) can be swapped in during a later phase without changing the surrounding architecture.

Step 3 — State classification. Maps the risk score to GREEN/YELLOW/ORANGE/RED using threshold bands, with the **multi-sensor confirmation rule** from Section 8.4 enforced: at least three independent sensor parameters must independently cross their thresholds before ORANGE or RED can be declared. This is the primary false-positive control and should be a first-class, testable rule in the implementation (not just a threshold on the aggregate score).

Step 4 — Simulated autonomous action. On entering ORANGE or RED, the fog node emits a structured, immutable **action log record** (JSON, append-only) representing what a real system would do — e.g., `{"action": "activate_mist_perimeter", "zone": "4B", "reason": [...], "timestamp": ...}`. No physical or simulated actuator is driven; this is a logging/reporting exercise that stands in for Section 4.3.2 Step 4 of the parent paper, and is the clearest "out of scope for hardware" boundary in the whole system.

Step 5 — Dual reporting. Publishes two message types on state change or on a periodic heartbeat:

- A short alert to the local "control center" feed (mirrors Report A in the parent paper).
- A full data packet (raw readings + score breakdown + decision log) to the cloud topic (mirrors Report B).

Step 6 — Cloud command handling. Subscribes to a command topic from the cloud container; if a command arrives (e.g., "raise sensitivity for zone 4B"), it adjusts its local thresholds. If no cloud connection is available, it continues operating on its last-known configuration — this is the specific behavior the offline-resilience test (Section 7) is designed to verify.

4.4 Fog–Fog Lateral Coordination

Implemented as an additional MQTT topic namespace (`region/lateral/{zone_id}`) that all fog nodes in a region subscribe to.

- When a fog node's risk score crosses YELLOW or higher, it publishes its zone ID, risk state, and current wind vector to the lateral channel.
- Neighboring fog nodes (configured with their adjacency and relative bearing to each other, matching the "zone map" concept in the parent paper) evaluate whether the reporting zone's wind direction points toward them. If so, they proactively raise their own internal alert level by one step (e.g., GREEN → YELLOW) even without their own sensors crossing a threshold — reproducing the predictive-alert behavior described in Section 5.3 and Section 6.3.
- This is the most novel logic path in the parent paper and the one most worth rigorously testing, since it has no real-world precedent to validate against.

4.5 Cloud Simulation Layer

Containers: `cloud-broker` (MQTT or lightweight Kafka), `timeseries-db` (TimescaleDB or InfluxDB), `dashboard` (Grafana).

- Ingests all fog-node reports; stores raw time-series and decision logs.
- Provides a simple synthetic "historical baseline" dataset per zone (a few seasons of fabricated normal-range data) so the risk model's seasonal/time-of-day terms have something to compare against — standing in for the parent paper's "correlate with FSI/NRSC" function (Section 4.4), without integrating any real satellite feed.
- Hosts a dashboard (Grafana) showing: current state per zone, a regional map view (static image with zone overlays is sufficient), decision logs, and latency metrics.

- Exposes a manual "advisory command" control (a simple form or API call) so a human operator can simulate the cloud sending a threshold-adjustment command to a fog node, exercising Step 6 above.

4.6 Scenario Injection Harness

Container/script: `scenario-injector`.

- A control script that drives the `edge-sim` containers through predefined trajectories via a simple control API or shared config, rather than requiring manual sensor manipulation.
- Scenario library (Section 7 below) is defined as versioned YAML files so scenarios are reproducible and can be re-run after any code change (regression testing).

4.7 Fault/Chaos Injection Layer

- Network partition: disconnect the `fog-node` container from the `cloud-broker` (via `docker network disconnect` or an `iptables` rule toggled by a script) to simulate the connectivity-loss conditions described in Section 2.3 and 8.2 of the parent paper.
- Sensor fault injection: handled inside `edge-sim` (Section 4.1) — stuck values, dropouts, out-of-range spikes.
- Multi-zone simultaneous escalation: run the scenario injector against several zones at once to test whether the fog/cloud layers handle concurrent high-risk events without cross-talk or dropped messages.

4.8 Control Center View

A minimal read-only panel (can be a Grafana panel or a small web page) showing exactly the information a Range Forest Officer would receive per the parent paper's Report A format: zone ID, risk level, key parameters, and simulated actions taken — deliberately kept simple, since the real control-center UX is out of scope for this simulation.

5. Docker Architecture

5.1 Container inventory

Container	Image basis	Role	Scale
edge-sim	python:slim	Synthetic sensor generator	N per zone (configurable, e.g. 5–10)
mqtt-broker-local	eclipse-mosquitto	Edge↔fog message bus (LoRa surrogate)	1 per zone
fog-node	python:slim or node:slim	Risk scoring, state machine, lateral coordination	1 per zone
cloud-broker	eclipse-mosquitto or bitnami/kafka	Fog↔cloud message bus	1 (shared)
timeseries-db	timescale/timescale db or influxdb	Historical + live data store	1 (shared)
dashboard	grafana/grafana	Visualization	1 (shared)
scenario-injector	python:slim	Drives test scenarios	1 (on demand)
chaos-controller	python:slim + iproute2	Network/sensor fault injection	1 (on demand)

All orchestrated via a single `docker-compose.yml`, with per-zone services parameterized using Compose's `deploy/environment-variable` pattern so adding a new simulated zone is a config change, not a code change.

5.2 Topic / message naming convention

zone/{zone_id}/edge/{node_id}/reading — edge sensor readings

zone/{zone_id}/fog/state — current fog risk state (heartbeat)

zone/{zone_id}/fog/alert — control-center alert (Report A)

zone/{zone_id}/fog/action_log — simulated autonomous action record
region/lateral/{zone_id} — lateral coordination broadcast
cloud/{zone_id}/report — full data packet to cloud (Report B)
cloud/{zone_id}/command — advisory command from cloud to fog

5.3 Data model sketch

Edge reading (JSON):

```
{  
  "node_id": "E12", "zone_id": "4B", "timestamp": "...",  
  "temperature_c": 34.2, "humidity_pct": 21.5,  
  "wind_speed_kmh": 14.0, "wind_dir_deg": 220,  
  "soil_moisture_pct": 9.1, "gas_ppm": 12,  
  "thermal_anomaly_c": 2.1, "light_lux": 41000,  
  "rain_mm": 0.0, "gps": [21.94, 86.32]  
}
```

Fog decision record (JSON):

```
{  
  "zone_id": "4B", "timestamp": "...",  
  "risk_score": 0.78, "state": "ORANGE",  
  "confirming_sensors": ["gas_ppm", "thermal_anomaly_c", "soil_moisture_pct"],  
  "actions_logged": ["activate_mist_perimeter", "notify_control_center"],  
  "cloud_connected": false
```

}

6. Simulated Test Scenarios

Scenario	What it exercises	Expected observable result
S1 — Normal day	Baseline operation	State stays GREEN; periodic cloud telemetry only
S2 — Slow-building risk	Gradual multi-parameter drift	State transitions GREEN → YELLOW correctly; sampling rate increases
S3 — Sudden ignition	Rapid multi-sensor spike	State reaches ORANGE/RED within seconds of the injected spike; action log populated
S4 — Single sensor fault	Fault-mode edge node	State does not escalate on a single faulty sensor alone (validates the 3-sensor confirmation rule)
S5 — Cloud outage during event	Chaos-controller cuts fog↔cloud link mid-scenario	Fog node continues operating and logging actions locally; queued reports flush once link is restored
S6 — Lateral spread prediction	S3 triggered in one zone with wind vector pointing at a neighbor	Neighboring zone's fog node raises its own state pre-emptively before its own sensors cross threshold
S7 — Concurrent multi-zone escalation	S3 run in 2+ zones simultaneously	No dropped messages, no state cross-contamination between zones, cloud dashboard reflects both correctly

7. Validation Metrics

Metric	How it's measured	What it stands in for
Fog decision latency	Timestamp delta: last confirming sensor reading → state-change event	The "30 seconds to 3 minutes" claim in Section 7.1 (fog-level portion only)
Lateral alert propagation time	Timestamp delta: source zone escalation → neighbor zone's pre-emptive state change	The predictive capability claimed in Section 6.3
False-positive rate under fault injection	Count of unwarranted ORANGE/RED states during S4-style runs across many trials	The multi-sensor confirmation design (Section 8.4)
Offline continuity	Whether fog decision-making and logging continue uninterrupted during S5	The offline-resilience claim (Sections 2.3, 8.2)
Concurrent-zone integrity	Message loss / cross-talk count during S7	Scalability of the message-bus design across multiple zones

These are the numbers worth reporting as evidence in your writeup — they are honest, measurable claims about the *software decision architecture*, not about the physical system.

8. Technology Stack Summary

- **Languages:** Python (edge simulators, fog logic, scenario/chaos scripts); optionally Node.js for the fog service if you prefer.
- **Messaging:** Eclipse Mosquitto (MQTT) for both edge↔fog and fog↔cloud tiers; Kafka as an optional upgrade if you want to demonstrate higher-throughput cloud ingestion.
- **Storage:** TimescaleDB or InfluxDB for time-series data.

- **Visualization:** Grafana for dashboards; a small static web page for the control-center view if Grafana feels too heavy for that piece.
 - **Orchestration:** Docker Compose (sufficient for this scale; no Kubernetes needed).
 - **Fault injection:** Linux `tc netem` and `iptables/docker network disconnect`, scripted via a small Python controller.
 - **Testing/reproducibility:** Scenario definitions as versioned YAML; a simple test runner that replays each scenario and asserts on the metrics in Section 7.
-

9. Development Phases (for the simulation itself)

Phase A — Core pipeline. Single edge node → single fog node → console log output. Validate the basic ingestion and rule-based scoring logic (no MQTT yet, direct function calls).

Phase B — Containerize and message-bus. Wrap components in Docker, introduce MQTT, support multiple edge nodes per zone, add the local control-center feed.

Phase C — Cloud layer. Add cloud broker, time-series DB, dashboard; implement dual reporting and the advisory-command path.

Phase D — Multi-zone and lateral coordination. Stand up 3+ simulated zones with adjacency/wind-bearing config; implement and test the lateral alert logic (S6).

Phase E — Fault and chaos testing. Implement the chaos-controller; run S4, S5, S7; collect and report the Section 7 metrics.

Phase F — Scenario library and reporting. Finalize the YAML scenario library, automate metric collection, and produce the results writeup that feeds back into the overall project report.

10. Explicit Scope Boundary (Simulation vs. Hardware)

Aspect	In scope for this simulation	Out of scope (deferred to hardware phase)
Sensor readings	Synthetic generation with realistic ranges and fault modes	Real sensor calibration, drift, physical fault behavior

Aspect	In scope for this simulation	Out of scope (deferred to hardware phase)
Edge↔fog communication	MQTT with optional artificial latency/loss	Real LoRa RF propagation, range, interference, antenna design
Fog decision logic	Fully implemented and testable	— (this is fully in scope and is the main deliverable)
Autonomous actions	Logged as structured records only	Real mist/sprinkler actuation, drone dispatch, acoustic deterrents
Power system	Not modeled	Solar sizing, battery chemistry, power-fail behavior
Fog↔fog / fog↔cloud links	MQTT with simulated outages	Real 4G/VSAT/sub-GHz RF link behavior
Governance/logging	Immutable action-log format implemented	Legal/regulatory sign-off process, DGCA drone compliance
Hardware durability	Not applicable	IP67 enclosures, temperature range, tamper detection
Institutional integration	Not applicable	RFO workflows, FSI/NRSC data-sharing agreements

11. Path Back to Hardware (Brief)

Once the simulation validates the decision architecture and produces stable metrics against the Section 7 targets, the natural next step — outside this project's scope — is the Phase 1 pilot described in the parent paper (Section 9.1): a small real deployment (5–10 edge nodes, 1 fog node) in a single forest division, in advisory-only mode, with the simulation's message schemas and risk-scoring config reused as-is so the transition from simulated to real fog logic is a swap of the data source, not a redesign.

12. Limitations of This Simulation Approach

- Cannot validate anything about RF propagation, power system sizing, or physical hardware durability — these require field testing regardless of how good the simulation is.
 - Synthetic sensor data, however carefully modeled, will not capture the full statistical texture of real forest conditions or real sensor failure modes; the false-positive rate measured in simulation is a lower bound, not a prediction of real-world performance.
 - The "autonomous action" outputs are logged intentions, not verified control of real systems; safety-critical behavior (e.g., does a real mist system actually activate correctly) is unverified until hardware integration.
 - Human-factors elements (would a Range Forest Officer actually trust and act on this alert format) cannot be tested in simulation.
-

13. Conclusion

This simulation is scoped to do one thing well: prove out the fog-level decision architecture — the composite risk model, the state machine, the multi-sensor confirmation rule, lateral coordination, and offline resilience — in a repeatable, containerized environment, producing quantitative evidence for the claims made in the parent hardware paper. It deliberately excludes anything requiring physical hardware, RF characterization, or power engineering, leaving those for a subsequent field-pilot phase once the software architecture is validated.